

AI ASSISTANT CODING ASSIGNMENT- 11.5

Name: M.sai charani

Enrollment no: 2303A52192

Batch: 34 CSE(AIML)

Lab 11 – Data Structures with AI: Implementing Fundamental Structures

Task Description #1 – Stack Implementation

Task: Use AI to generate a Stack class with push, pop, peek, and is_empty methods.

Sample Input Code: class Stack: pass

Expected Output:

- A functional stack implementation with all required methods and docstrings.

```
"""Prompt:
Generate a Python class named Stack that implements the stack data
structure using a list. The class should include the following
methods:
push(item) – to add an element to the top of the stack
pop() – to remove and return the top element
peek() – to return the top element without removing it
is_empty() – to check whether the stack is empty
Include proper docstrings for the class and each method. Handle edge
cases such as popping from an empty stack."""
```

```
main.py
--
10 class Stack:
11     """
12     A class to represent a stack data structure.
13     Uses a Python list to store stack elements.
14     """
15     def __init__(self):
16         """Initialize an empty stack."""
17         self.items = []
18
19     def push(self, item):
20         """
21         Add an element to the top of the stack.
22         :param item: Element to be added to the stack
23         """
24         self.items.append(item)
25
26     def pop(self):
27         """
28         Remove and return the top element of the stack.
29         :return: Top element of the stack
30         :raises IndexError: If the stack is empty
31         """
32         if self.is_empty():
33             raise IndexError("Pop from empty stack")
34         return self.items.pop()
35
36     def peek(self):
37         """
38         Return the top element without removing it.
39         :return: Top element of the stack
40         :raises IndexError: If the stack is empty
41         """
42         if self.is_empty():
43             raise IndexError("Peek from empty stack")
44         return self.items[-1]
45
46     def is_empty(self):
47         """
48         Check whether the stack is empty.
49         :return: True if stack is empty, False otherwise
50         """
51         return len(self.items) == 0
52
53
54 if __name__ == "__main__":
55     stack = Stack()
56
57     stack.push(10)
58     stack.push(20)
59     stack.push(30)
60
61     print("Top element:", stack.peek()) # 30
62     print("Popped element:", stack.pop()) # 30
63     print("Is stack empty?", stack.is_empty()) # False
```

Output

```
Top element: 30
Popped element: 30
Is stack empty? False

=== Code Execution Successful ===
```

Task Description #2 – Queue Implementation Task:

Use AI to implement a Queue using Python lists.

Sample Input Code:

class Queue:

pass

Expected Output:

- FIFO-based queue class with enqueue, dequeue, peek, and size methods.

""" Prompt:

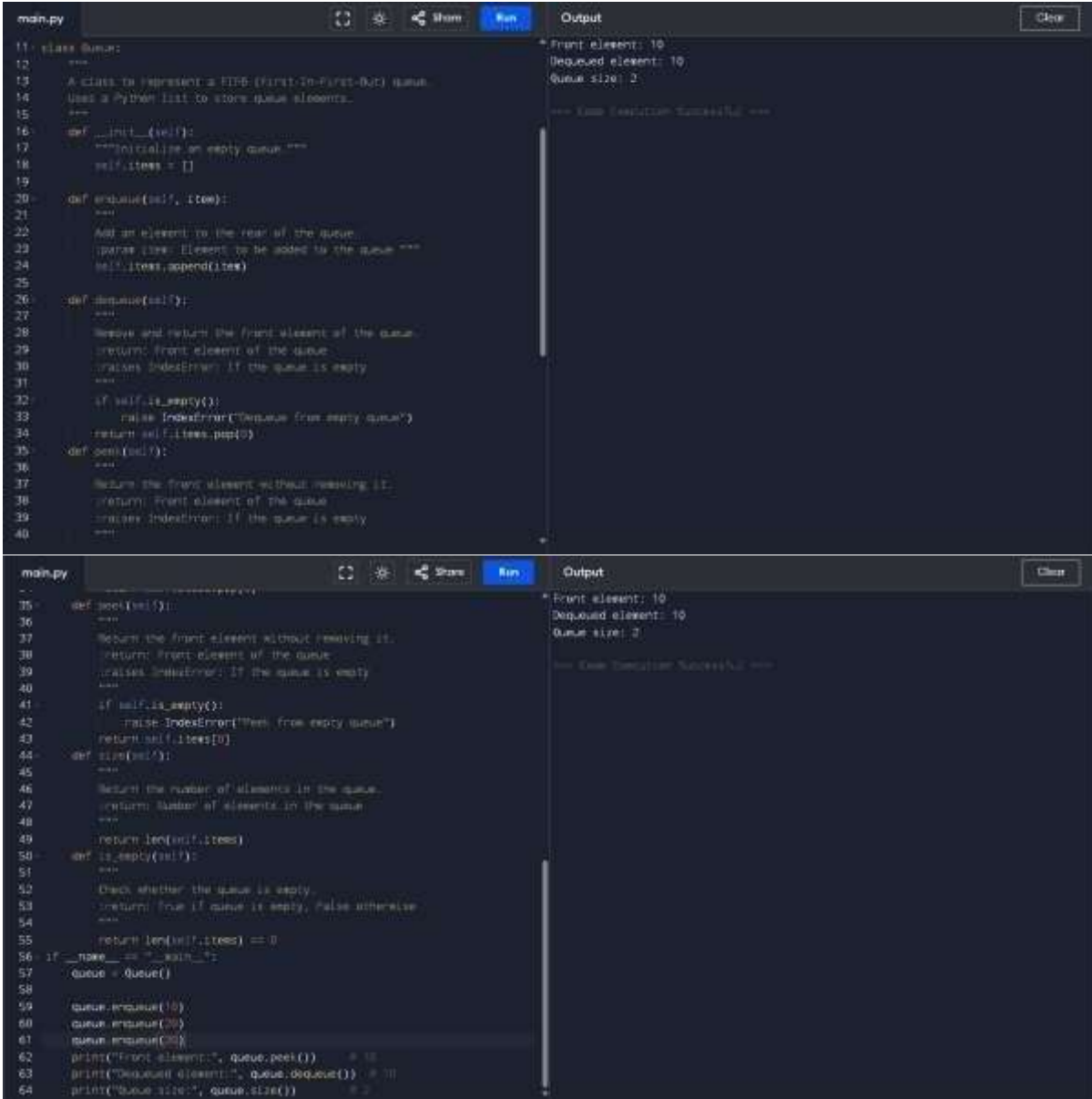
Generate a Python class named Queue that implements a FIFO (First-In-First-Out) queue using a Python list. The class should include the following methods:

enqueue(item) - to add an element to the rear of the queue

dequeue() - to remove and return the front element

peek() - to return the front element without removing it

size() - to return the number of elements in the queue """



```
main.py 11: class Queue:
12:     """
13:     A class to represent a FIFO (First-In-First-Out) queue.
14:     Uses a Python list to store queue elements.
15:     """
16:     def __init__(self):
17:         """Initialize an empty queue"""
18:         self.items = []
19:
20:     def enqueue(self, item):
21:         """
22:         Add an element to the rear of the queue.
23:         param item: Element to be added to the queue
24:         """
25:         self.items.append(item)
26:
27:     def dequeue(self):
28:         """
29:         Remove and return the front element of the queue.
30:         returns: Front element of the queue
31:         raises IndexError: If the queue is empty
32:         """
33:         if self.is_empty():
34:             raise IndexError("Dequeue from empty queue")
35:         return self.items.pop(0)
36:
37:     def peek(self):
38:         """
39:         Return the front element without removing it.
40:         returns: Front element of the queue
41:         raises IndexError: If the queue is empty
42:         """
43:
44:     def size(self):
45:         """
46:         Return the number of elements in the queue.
47:         returns: Number of elements in the queue
48:         """
49:         return len(self.items)
50:
51:     def is_empty(self):
52:         """
53:         Check whether the queue is empty.
54:         returns: True if queue is empty, False otherwise
55:         """
56:         return len(self.items) == 0
57:
58: if __name__ == "__main__":
59:     queue = Queue()
60:
61:     queue.enqueue(10)
62:     queue.enqueue(20)
63:     queue.enqueue(30)
64:
65:     print("Front element:", queue.peek()) # 10
66:     print("Dequeued element:", queue.dequeue()) # 10
67:     print("Queue size:", queue.size()) # 2
```

Output

```
* Front element: 10
* Dequeued element: 10
* Queue size: 2
*** Code Execution Successful ***
```

Task Description #3 – Linked List

Task: Use AI to generate a Singly Linked List with insert and display methods.

Sample Input Code:

class Node: pass

class LinkedList:

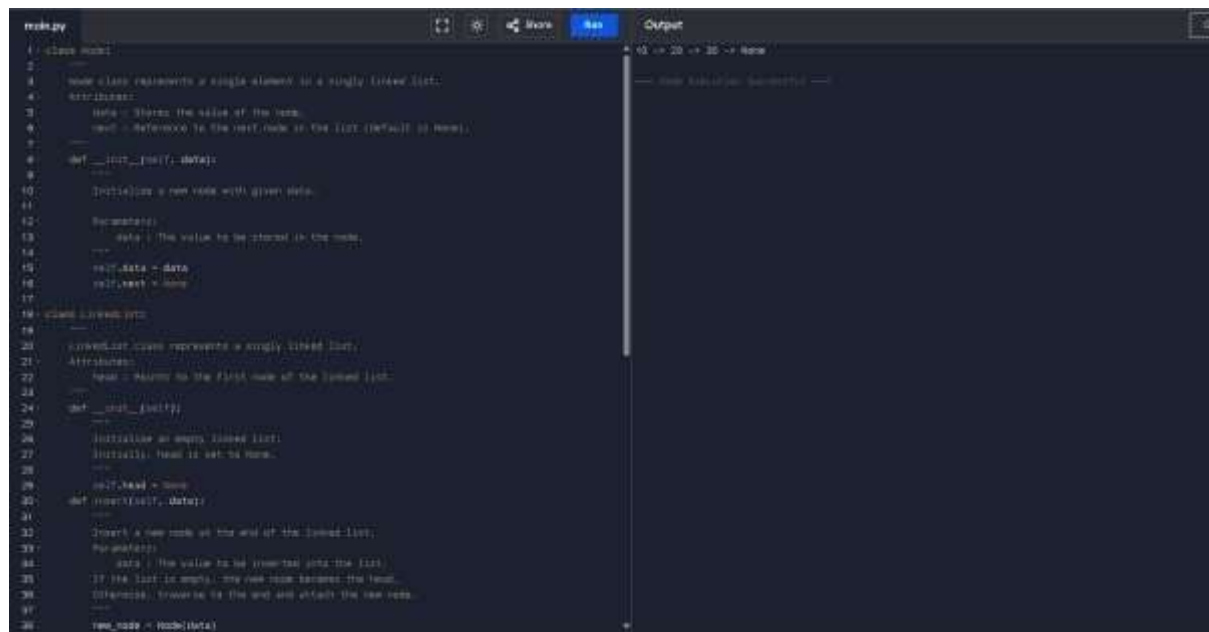
pass

Expected Output:

- A working linked list implementation with clear method documentation

```
""" Prompt:
Generate a Python implementation of a Singly Linked List.
Create a Node class to store data and a reference to the next node.
Create a LinkedList class with the following methods:

insert(data) - to insert a new node at the end of the list
display() - to display all elements of the linked list
Include clear docstrings for all classes and methods. """
```



```
1 class Node:
2     """
3     Node class represents a single element in a singly linked list.
4     Attributes:
5         data: Stores the value of the node.
6         next: Reference to the next node in the list (default is None).
7     """
8     def __init__(self, data):
9         """
10        Initialize a new node with given data.
11        """
12        Parameters:
13            data: The value to be stored in the node.
14        """
15        self.data = data
16        self.next = None
17
18 class LinkedList:
19     """
20     LinkedList class represents a singly linked list.
21     Attributes:
22         head: Points to the first node of the linked list.
23     """
24     def __init__(self):
25         """
26        Initialize an empty linked list.
27        Initially, head is set to None.
28        """
29        self.head = None
30
31     def insert(self, data):
32         """
33        Insert a new node at the end of the linked list.
34        Parameters:
35            data: The value to be inserted into the list.
36        """
37        If the list is empty, the new node becomes the head.
38        Otherwise, traverse to the end and attach the new node.
39
40        """
41        new_node = Node(data)
```

```

44 data = The value to be inserted into the list.
45 If the list is empty, the new node becomes the head.
46 Otherwise, traverse to the end and attach the new node.
47
48 new_node = Node(data)
49 # If list is empty, new node becomes the head
50 if self.head is None:
51     self.head = new_node
52     return
53 # Traverse to the last node
54 current = self.head
55 while current.next:
56     current = current.next
57 # Link the last node to the new node
58 current.next = new_node
59 self.display_list()
60
61
62 Display all elements of the linked list.
63 Traverse from head to the last node
64 and prints each element followed by '\n'
65
66 current = self.head
67 if current is None:
68     print('The list is empty.')
69     return
70 while current:
71     print(current.data, end=" ")
72     current = current.next
73
74 print("\n")
75
76 if __name__ == '__main__':
77     linked_list = LinkedList()
78     linked_list.insert(10)
79     linked_list.insert(20)
80     linked_list.insert(30)
81     linked_list.insert(40)
82     linked_list.display()

```

Task Description #4 – Hash Table

Task: Use AI to implement a hash table with basic insert, search, and delete methods.

Sample Input Code: class HashTable:

pass

Expected Output:

- Collision handling using chaining, with well-commented methods

```

1 """Task Description #4 - Hash Table
2 Task: Use AI to implement a hash table with basic insert, search, and delete
3 methods.
4 Sample Input Code:
5 class HashTable:
6     pass
7 Expected Output:
8 • Collision handling using chaining, with well-commented methods. """

```

```

1 class Hashtable:
2     """Hash table implementation using chaining for collision handling.
3     Attributes:
4         size: Size of the hash table.
5         table: List of buckets (each bucket is a list).
6     """
7
8     def __init__(self, size=10):
9         """
10         Initialize the hash table with given size.
11         Parameters:
12             size (int): Number of buckets in the hash table.
13         """
14         self.size = size
15         self.table = [[] for _ in range(size)]
16
17     def _hash(self, key):
18         """
19         Private method to compute hash index for a given key.
20         Parameters:
21             key: Key to hash.
22         Returns:
23             int: Hashed key value.
24         """
25         return hash(key) % self.size
26
27     def insert(self, key, value):
28         """
29         Insert a key-value pair into the hash table.
30         Handles collisions using chaining.
31         If the key already exists, update its value.
32         """
33         index = self._hash(key)
34         bucket = self.table[index]
35         for pair in bucket:
36             if pair[0] == key:
37                 pair[1] = value
38                 return
39         bucket.append([key, value])
40
41     def search(self, key):
42         """
43         Search for a key in the hash table.
44         Parameters:
45             key: Key to search.
46         Returns:
47             value: Value associated with key, or None if not found.
48         """
49         index = self._hash(key)
50         bucket = self.table[index]
51         for pair in bucket:
52             if pair[0] == key:
53                 return pair[1]
54         return None
55
56     def delete(self, key):
57         """
58         Delete a key-value pair from the hash table.
59         Parameters:
60             key: Key to delete.
61         Returns:
62             bool: True if deleted successfully, False if key not found.
63         """
64         index = self._hash(key)
65         bucket = self.table[index]
66         for i, pair in enumerate(bucket):
67             if pair[0] == key:
68                 del bucket[i]
69                 return True
70         return False
71
72 if __name__ == "__main__":
73     ht = Hashtable()
74     ht.insert("name", "John")
75     ht.insert("age", 25)
76     ht.insert("city", "New York")
77     print("Search name:", ht.search("name"))
78     print("Delete age:", ht.delete("age"))
79     print("Search age:", ht.search("age"))

```

Task Description #5 – Graph Representation

Task: Use AI to implement a graph using an adjacency list.

Sample Input Code:

```
class Graph:
```

```
pass
```

Expected Output:

- Graph with methods to add vertices, add edges, and display connections.

```
- """ Prompt:
2 Generate a Python class named Graph that represents a graph using an adjacency list.
3 The class should include the following methods:
4 add_vertex(vertex) - to add a new vertex to the graph
5 add_edge(vertex1, vertex2) - to add an edge between two vertices
6 display() - to display all vertices and their connected neighbors. """

# Main.py
1 class Graph:
2     """
3     Graph implementation using an adjacency list.
4     Attributes:
5     adj_list: Dictionary mapping vertices to lists
6     """
7     def __init__(self):
8         """Initialize an empty graph"""
9         self.adj_list = {}
10
11     def add_vertex(self, vertex):
12         """Add a vertex to the graph"""
13         self.adj_list[vertex] = []
14
15     def add_edge(self, vertex1, vertex2):
16         """Add an edge between two vertices"""
17         self.adj_list[vertex1].append(vertex2)
18         self.adj_list[vertex2].append(vertex1)
19
20     def display(self):
21         """Display the graph"""
22         for vertex in self.adj_list:
23             print(f"Vertex: {vertex}, Neighbors: {self.adj_list[vertex]}")
24
25 # Test the Graph class
26 graph = Graph()
27 graph.add_vertex(1)
28 graph.add_vertex(2)
29 graph.add_vertex(3)
30 graph.add_edge(1, 2)
31 graph.add_edge(2, 3)
32 graph.add_edge(3, 1)
33 graph.display()
```

Task Description #6: Smart Hospital Management System – Data Structure Selection A

hospital wants to develop a Smart Hospital Management System that handles:

1. Patient Check-In System – Patients are registered and treated in order of arrival.
2. Emergency Case Handling – Critical patients must be treated first.
3. Medical Records Storage – Fast retrieval of patient details using ID.
4. Doctor Appointment Scheduling – Appointments sorted by time.
5. Hospital Room Navigation – Represent connections between wards and rooms.

Student Task:

- For each feature, select the most appropriate data structure from the list below: o

Stack o Queue

o Priority

Queue o

Linked List

o Binary Search Tree (BST) o Graph o Hash

Table o Deque

- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Smart Hospital Management System – Data Structure Selection

1. Patient Check-In System Selected

Data Structure: Queue

Justification:

A Queue follows the First-In-First-Out (FIFO) principle. Patients are treated in the order they arrive, ensuring fairness. Therefore, a Queue is the most suitable structure for managing regular patient check-ins.

2. Emergency Case Handling

Selected Data Structure: Priority Queue

Justification:

Emergency patients must be treated based on severity rather than arrival time. A Priority Queue allows higher-priority (critical) patients to be treated first. This ensures immediate attention to life-threatening cases.

3. Medical Records Storage Selected Data Structure: Hash

Table

Justification:

Patient records need fast retrieval using a unique patient ID. A Hash Table provides average $O(1)$ time complexity for search, insert, and delete operations. This makes record management efficient and scalable.

4. Doctor Appointment Scheduling

Selected Data Structure: Binary Search Tree (BST)

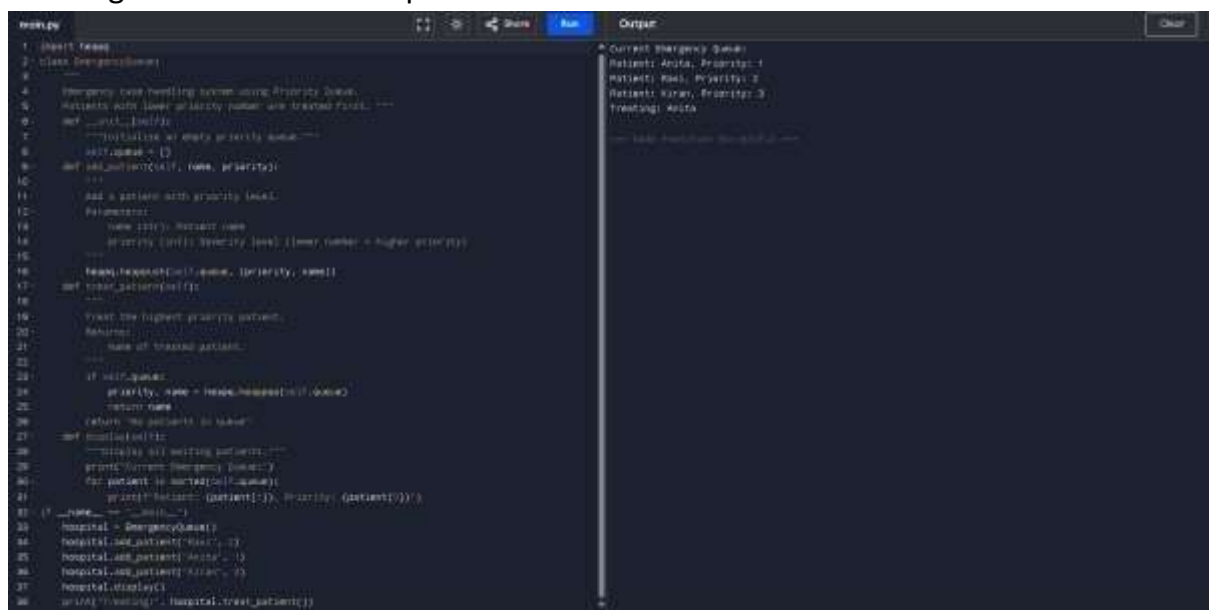
Justification:

Appointments must be maintained in sorted order based on time. A Binary Search Tree automatically keeps elements sorted. This makes scheduling and time-based retrieval efficient.

5. Hospital Room Navigation Selected Data Structure: Graph

Justification:

Hospital rooms and wards can be represented as nodes, and pathways as edges. A Graph effectively models connections between locations. It also supports pathfinding for navigation within the hospital.



```
1 class EmergencyQueue:
2     def __init__(self):
3         self.queue = []
4         self.priority_queue = []
5         self.current_patient = None
6         self.current_priority = None
7         self.current_name = None
8         self.current_waiting_time = None
9         self.current_waiting_time_queue = []
10
11     def add_patient(self, name, priority):
12         """Add a patient with priority level.
13         Parameters:
14             name (str): Patient name
15             priority (int): Priority level (lower number = higher priority)
16         """
17         self.queue.append((name, priority))
18         self.priority_queue.append((name, priority))
19
20     def remove_highest_priority_patient(self):
21         """Remove the highest priority patient.
22         Returns:
23             name (str): Name of highest priority patient
24         """
25         if self.queue:
26             name, priority = self.queue.pop(0)
27             return name
28         return "No patients to remove"
29
30     def display(self):
31         """Display all waiting patients.
32         Prints:
33             Current Emergency Queue:
34             Patient: [name], Priority: [priority]
35             Patient: [name], Priority: [priority]
36             Patient: [name], Priority: [priority]
37         """
38         if self.queue:
39             print("Current Emergency Queue:")
40             for patient in self.queue:
41                 print(f"Patient: {patient[0]}, Priority: {patient[1]}")
42         else:
43             print("No patients in queue")
44
45     def __str__(self):
46         return f"EmergencyQueue: {self.queue}"
47
48     def __repr__(self):
49         return f"EmergencyQueue: {self.queue}"
50
51     def __len__(self):
52         return len(self.queue)
53
54     def __getitem__(self, index):
55         return self.queue[index]
56
57     def __setitem__(self, index, value):
58         self.queue[index] = value
59
60     def __delitem__(self, index):
61         del self.queue[index]
```

Task Description #7: Smart City Traffic Control System A city plans

a Smart Traffic Management System that includes:

1. Traffic Signal Queue – Vehicles waiting at signals.
2. Emergency Vehicle Priority Handling – Ambulances and fire trucks prioritized.
3. Vehicle Registration Lookup – Instant access to vehicle details.
4. Road Network Mapping – Roads and intersections connected logically.
5. Parking Slot Availability – Track available and occupied slots.

Student Task

- For each feature, select the most appropriate data structure from the list below: o

Stack o Queue

o Priority Queue o Linked List

o Binary Search

Tree (BST) o Graph o

Hash Table o Deque

- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

Smart City Traffic Control System

1. Traffic Signal Queue Selected Data

Structure: Queue

Justification:

Vehicles waiting at a traffic signal follow the First-In-First-Out (FIFO) principle. The vehicle that arrives first should move first when the signal turns green. Therefore, a Queue is the most appropriate structure for managing traffic signals fairly and efficiently.

2. Emergency Vehicle Priority Handling Selected Data Structure: Priority Queue

Justification:

Emergency vehicles such as ambulances and fire trucks must be given immediate access regardless of arrival time. A Priority Queue allows higher-priority vehicles to be processed before regular vehicles. This ensures faster emergency response and improved public safety.

3. Vehicle Registration Lookup

Selected Data Structure: Hash Table

Justification:

Vehicle details must be accessed instantly using a registration number. A Hash Table provides average $O(1)$ time complexity for search operations. This makes it ideal for fast and efficient vehicle information retrieval.

4. Road Network Mapping Selected

Data Structure: Graph

Justification:

Roads and intersections can be represented as nodes and edges. A Graph efficiently models connections between locations and supports route-finding algorithms. This makes it suitable for navigation and traffic management systems.

5. Parking Slot Availability

Selected Data Structure: Binary Search Tree (BST)

Justification:

Parking slots can be stored in sorted order based on slot numbers. A Binary Search Tree maintains elements in sorted order and allows efficient insertion, deletion, and searching. This makes it suitable for tracking available and occupied parking slots.

```

main.py
1 class Stack:
2     """
3     Stack implementation using Python list.
4     follows LIFO (Last-In-First-Out) principle. """
5     def __init__(self):
6         """
7         Initializes an empty stack.
8         """
9         self.items = []
10    def push(self, item):
11        """
12        Add an element to the top of the stack.
13        Parameters:
14        item: Element to be added
15        """
16        self.items.append(item)
17    def pop(self):
18        """
19        Remove and return the top element of the stack.
20        Returns:
21        The top element of the stack.
22        Raises:
23        IndexError: If stack is empty. """
24        if self.is_empty():
25            raise IndexError("Pop from empty stack")
26        return self.items.pop()
27    def peek(self):
28        """
29        Return the top element without removing it.
30        Returns:
31        The top element of the stack.
32        Raises:
33        IndexError: If stack is empty. """
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

```

main.py
27 def peek(self):
28     """
29     Return the top element without removing it.
30     Returns:
31     The top element of the stack.
32     Raises:
33     IndexError: If stack is empty. """
34     if self.is_empty():
35         raise IndexError("Pop from empty stack")
36     return self.items[-1]
37 def is_empty(self):
38     """
39     Check whether the stack is empty.
40     Returns:
41     True if stack is empty, otherwise False.
42     """
43     return len(self.items) == 0
44 def size(self):
45     """
46     Return the number of elements in the stack.
47     """
48     return len(self.items)
49 if __name__ == "__main__":
50     stack = Stack()
51
52     stack.push(10)
53     stack.push(20)
54     stack.push(30)
55
56     print("Top element:", stack.peek())
57     print("Popped element:", stack.pop())
58     print("Stack size:", stack.size())
59     print("Is stack empty?", stack.is_empty())

```

```

Output
* Top element: 30
* Popped element: 30
* Stack size: 2
* Is stack empty? False

--- Code Execution Successful ---

* Top element: 30
* Popped element: 30
* Stack size: 2
* Is stack empty? False

--- Code Execution Successful ---

```

Task Description #8: Smart E-Commerce Platform – Data Structure.

Challenge

An e-commerce company wants to build a Smart Online Shopping System with:

1. Shopping Cart Management – Add and remove products dynamically.
2. Order Processing System – Orders processed in the order they are Placed.
3. Top-Selling Products Tracker – Products ranked by sales count.
4. Product Search Engine – Fast lookup of products using product ID.

5. Delivery Route Planning – Connect warehouses and delivery locations.

Student Task

- For each feature, select the most appropriate data structure from the list below:

- o Stack o Queue
- o Priority Queue o Linked List
- o Binary Search

Tree (BST) o Graph o

Hash Table o Deque

- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

Smart E-Commerce Platform – Data Structure Selection

1. Shopping Cart Management

Selected Data Structure: Linked List

Justification:

Products in a shopping cart are added and removed dynamically. A Linked List allows efficient insertion and deletion without shifting elements. This makes it suitable for managing frequently changing cart items.

2. Order Processing System Selected

Data Structure: Queue

Justification:

Orders must be processed in the order they are placed. A Queue follows the FirstInFirst-Out (FIFO) principle, ensuring fairness. Therefore, it is ideal for handling customer orders.

3. Top-Selling Products Tracker

Selected Data Structure: Priority

Queue

Justification:

Products need to be ranked based on sales count. A Priority Queue allows items with higher sales to be accessed first. This ensures efficient tracking of top-selling products.

4. Product Search Engine

Selected Data Structure: Hash Table

Justification:

Products are searched using unique product IDs. A Hash Table provides average $O(1)$ time complexity for search operations. This makes product lookup fast and efficient.

5. Delivery Route Planning Selected

Data Structure: Graph

Justification:

Warehouses and delivery locations can be represented as nodes, and roads as edges. A Graph efficiently models connections and supports shortest path algorithms. This makes it suitable for route optimization and logistics planning.

```

1 # Create a Product class
2 # Attributes: Product Name, Price, Stock, Item Type
3 # Methods: Add to Stock, Product Search
4 products = {}
5
6 # Function to add a product
7 def add_product(product_id, name, price):
8     products[product_id] = {
9         'name': name,
10        'price': price
11    }
12
13 # Function to add product successfully
14 print("Product added successfully.")
15
16 # Function to search for a product
17 def search_product(product_id):
18     if product_id in products:
19         print("Product Found:")
20         print("Product ID:", product_id)
21         print("Name:", products[product_id]['name'])
22         print("Price:", products[product_id]['price'])
23     else:
24         print("Product not found.")
25
26 # Function to display all products
27 def display_products():
28     if len(products) == 0:
29         print("No products available.")
30     else:
31         print("Available Products:")
32         for pid in products:
33             print(pid, ":", "Name:", products[pid]['name'], "Price:", products[pid]['price'])
34
35 # Main Function
36 add_product(101, 'Laptop', 95000)
37 add_product(102, 'Mobile', 20000)
38 add_product(103, 'Headphones', 3000)
39 print()
40 display_products()
41
42 # Searching for Product 102
43 search_product(102)
44
45 # Searching for Product 200
46 search_product(200)
47
48 # Product not found
49 search_product(104)
50
51 # End of Program. Successful.
```